

LAB 2: System Calls

Saman Waseem 29258

```
cat > answers-syscall.txt << 'EOF'
```

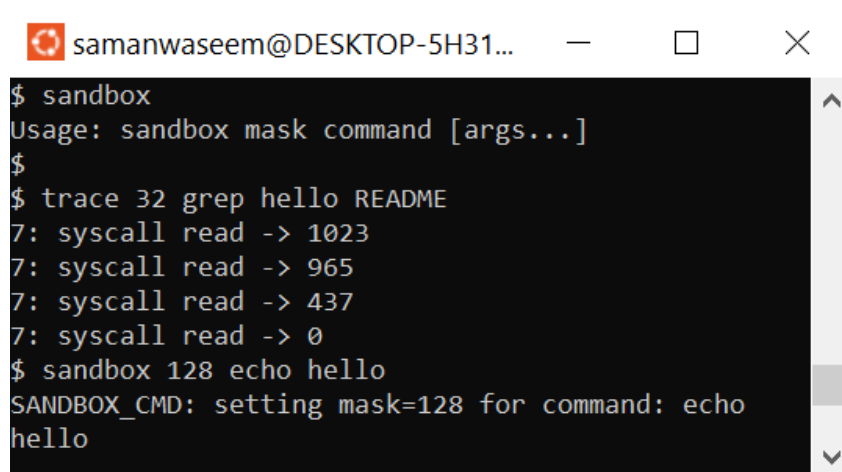
1. The main goal of this lab is to understand how system calls work in xv6 by implementing new system calls (trace and sandbox) and modifying the system call handler.

2. The trace system call works by setting a bitmask in the process structure. When a system call is made, if the corresponding bit is set in the trace mask, information about that system call is printed.

3. The sandbox system calls work by restricting which programs can be executed (command sandbox) or which file paths can be accessed (path sandbox). The restrictions are enforced in the exec system call and file system calls respectively.

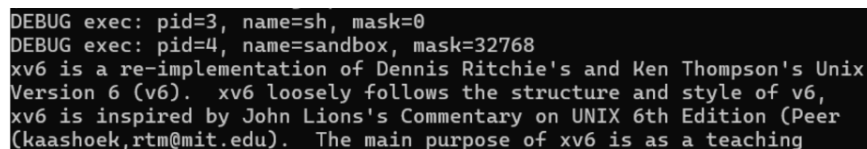
4. The most challenging part was modifying the system call handler to properly intercept and restrict system calls while maintaining compatibility with existing functionality. I encountered a lot of errors.

EOF



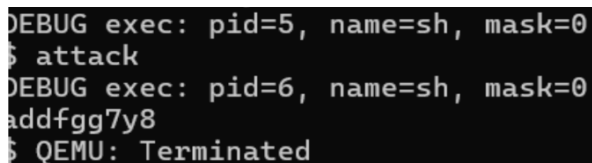
```
$ sandbox
Usage: sandbox mask command [args...]
$
$ trace 32 grep hello README
7: syscall read -> 1023
7: syscall read -> 965
7: syscall read -> 437
7: syscall read -> 0
$ sandbox 128 echo hello
SANDBOX_CMD: setting mask=128 for command: echo
hello
```

trace



```
DEBUG exec: pid=3, name=sh, mask=0
DEBUG exec: pid=4, name=sandbox, mask=32768
xv6 is a re-implementation of Dennis Ritchie's and Ken Thompson's Unix
Version 6 (v6). xv6 loosely follows the structure and style of v6,
xv6 is inspired by John Lions's Commentary on UNIX 6th Edition (Peer
(kaashoek,rtm@mit.edu)). The main purpose of xv6 is as a teaching
```

sandbox



```
DEBUG exec: pid=5, name=sh, mask=0
$ attack
DEBUG exec: pid=6, name=sh, mask=0
addfgg7y8
$ QEMU: Terminated
```

attack